Programación orientada a objetos

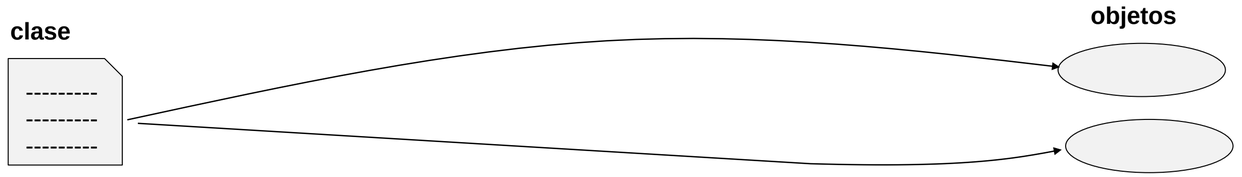
Conceptos generales

Clases y objetos

Un lenguaje de programación orientado a objetos no es solo aquel que utiliza objetos. Existen una serie de conceptos sobre los que se apoya la programación orientada a objetos y que permiten reutilizar y optimizar el código a la hora de desarrollar aplicaciones.

Pero antes de entrar en estos conceptos más específicos de la Programación Orientada a Objetos, aclaremos los conceptos base como son el de **clase** y **objeto**:

- **Clase**. Una clase define el comportamiento de un tipo de objeto, a través de la definición de **métodos y atributos** asociados a los mismos. Un método no es más que una función que define el comportamiento de un objeto de clase, mientras que un atributo representa una variable que almacena una propiedad del objeto. Los métodos siempre definen el parámetro **self**.
- **Objeto**. Si la clase contiene el código que define el comportamiento de un tipo de objeto, éste representa una instancia de dicha clase que puede utilizarse dentro del código para llamar a las funciones o métodos y establecer valores en los atributos del objeto. Los objetos se manejan mediante de variables



Definición de una clase

```
class Primeraclase:
    color="Amarillo"
    def imprimir(self):
        print(self.color)
```

Creación de objeto

```
pr=Primeraclase()
pr.color="Naranja"
pr.imprimir()
```

Atributos y métodos

En el interior de una clase encontramos **atributos**, que no son más que variables donde se almacenan propiedades de los objetos de la clase, y **métodos**, que son funciones para realizar tareas asociadas a los objetos de la clase.

Si desde el interior de un método se necesita acceder a alguno de los atributos, se utiliza la palabra reservada **self**, que se declara como parámetro en el método. self es una referencia al propio objeto y es **obligatorio que este parámetro esté definido** en todos los métodos de la clase, parámetro que es pasado implícitamente

```
class Calculadora:
    x=0
    y=0
    def sumar(self):
        print(self.x+self.y)
```

referencia al propio objeto para acceder a atributos

La creación de un objeto de la clase, se realiza asignando el nombre de la clase a una variable:

```
cal=Calculadora()
```

Con la variable objeto podemos utilizar el operador . para acceder a atributos y llamar a los métodos:

```
cal.x=24
cal.y=5
cal.sumar() # 29
```

Constructores

Una clase puede definir un constructor, que no es más que un bloque de código que se ejecuta automáticamente al crear un objeto de la clase. Se utilizan para inicializar atributos durante la creación del objeto.

Un constructor se define con la función especial **__init__()** y puede recibir parámetros que se usarán, habitualmente, para la inicialización de atributos:

```
class Calculadora:
    x=0
    y=0
    def __init__(self,a,b):
        self.x=a
        self.y=b
    def sumar(self):
        print(self.x+self.y)
```

En la creación del objeto se le proporcionan los parámetros al constructor:

```
cal=Calculadora(3,7)
cal.sumar() # 10
```

Al igual que los métodos, los constructores también pueden definir parámetros opcionales:

```
def __init__(self,a,b=3):
```

Herencia

Definición de herencia y uso práctico

La herencia es una de las características más interesantes de los lenguajes orientados a objetos, ya que permite crear nuevas clases a partir de clases ya existentes, sin tener que repetir código.

Cuando una clase debe incluir los métodos de otra, incorporando algunos adicionales, la nueva clase heredará la ya existente, adquiriendo automáticamente todos los métodos y atributos de ésta. Simplemente, se tendrán que codificar los nuevos métodos.

Para crear una clase que herede a otra, se indica entre paréntesis el nombre de la clase se hereda:

```
class Cientifica(Calculadora):
    def sumatorio(self):
        suma=0
        for c in range(self.x,self.y+1):
            suma+=c
        return suma
```

La clase hija hereda el constructor de la clase padre

```
cien=Cientifica(7,9)
print(cien.sumatorio())  #24
```

Constructores y herencia

Como hemos indicado anteriormente, una clase hija hereda también el constructor de la clase padre, aunque puede definir el suyo propio y anular al del padre

```
class Calculadora:
    x=0
    y=0
    def __init__(self,a,b):
        :
```

```
class Cientifica(Calculadora):
    pass
```

```
class Calculadora:
    x=0
    y=0
    def __init__(self,a,b):
        :
```

```
class Cientifica(Calculadora):
    def __init__(self, a, b):
        self.x=a+1
        self.y=b+4
```

Si se define un constructor en la clase hija, **solo podrá usarse ese constructor** para crear objetos de dicha clase.

En este caso se utiliza el constructor heredado de la clase padre Calculadora



```
cien=Cientifica(2,7)
```

En este caso se hace uso del constructor definido en la clase Cientifica



```
cien=Cientifica(4,1)
```

Herencia múltiple

Python permite la herencia múltiple, que consiste en crear clases que hereden más de una clase

Para heredar más de una clase, se indican entre paréntesis y separadas por una coma la lista de clases padre a heredar:

```
class Una:
    def m(self):
        print("una")
class Dos:
    def m(self):
        print("dos")
class Nueva(Una,Dos):
    pass
```

Si se heredan dos versiones de un mismo método desde dos clases padre, **prevalece el de la primera clase** de la lista

```
n=Nueva()
n.m() # imprime una
```

Sobrescritura de métodos

Concepto de sobrescritura

Cuando creamos una clase que hereda otra, es posible que alguno de los métodos heredados de la clase padre no se ajuste a las necesidades de la clase hija. En ese caso, la clase hija puede redefinir de nuevo el método heredado, creando uno nuevo con el **mismo nombre**.

```
class Circulo:
    radio=0
    def __init__(self,r):
        self.radio=r
    def superficie(self):
        return 3.1416*self.radio*self.radio
```

Si queremos definir una clase llamada Esfera que herede Circulo, habrá que definir de nuevo el método superficie para ajustarlo a la nueva clase:

```
class Esfera(Circulo):
    def superficie(self):
        return 4* 3.1416*self.radio*self.radio
```

Al crear un objeto de Esfera se utilizará la nueva versión del método superficie(), la versión definida en Circulo queda oculta para esta clase

```
n=Esfera(3)
print(n.superficie()) #113.1
```

Uso de super

Mediante la palabra reservada super podemos llamar desde una clase hija a métodos y atributos definidos en la clase padre. Para ello utilizamos la sintaxis:

super().metodo()

Esto es especialmente útil cuando desde un método que estamos sobrescribiendo en la clase hija queremos llamar a la versión original del método definida en la clase padre, evitando así la repetición de instrucciones.

Por ejemplo, en la clase Esfera podemos llamar al método *superficie()* de Circulo para no tener que escribir de nuevo el código:

```
class Circulo:
    radio=0
    def __init__(self,r):
        self.radio=r
    def superficie(self):
        return 3.1416*self.radio*self.radio
class Esfera(Circulo):
    def superficie(self):
        return 4*super().superficie()
```